

Stos

Liniowa struktura do przechowywania danych, w której mamy tylko dostęp do ostatnio dołożonej do niej wartości (położonej na jego "czubku" (top)).

Założmy, że mamy w zmiennej `stack` nasz stos. Aby manipulować jego zawartością w programie, potrzebujemy 3 zasadnicze instrukcje:

- `IS_EMPTY(stack)` -- zwraca `TRUE`, jeśli zawartość stosu jest pusta, `FALSE` inaczej.
- `POP(stack)` -- zwraca element na "czubku" stosu (`top`) i ściąga go ze stosu (usuwa go)
- `PUSH(stack, x)` - umieszcza `x` na "czubku" stosu.

Można też zdefiniować operację `check(stack)` - zwraca co jest na czubku stosu, ale nie usuwa tego elementu (ale można to uzyskać tak samo poprzez `x=POP(stack); PUSH(stack,x), RETURN x`).

Implementacja stosu

Można powiedzieć, że żyje poziom wyżej w świecie abstrakcji niż połączona lista jednokierunkowa czy tablica. Możemy go zaimplementować używając tych dwóch typów.

Stos oparty na tablicy

Jeśli operujemy tablicą statyczną, ma swój określony, maksymalny rozmiar. Musimy pamiętać indeks elementu tablicy, który odpowiada elementowi na "czubku" stosu i odpowiednio modyfikować tablicę według zadanej logiki. np.:

```
inicjalizacja stosu o rozmiarze 5 przechowywania liczb >=0 :
[-1,-1,-1,-1,-1]
IS_EMPTY(stack) // da nam TRUE
PUSH(stack,3)
[3,-1,-1,-1,-1]
PUSH(stack,4)
[3,4,-1,-1,-1]
IS_EMPTY(STACK) // da FALSE
x=POP(stack)    // x =4
[3,-1,-1,-1,-1]
x=POP(stack)    //x =3
[-1,-1,-1,-1,-1]
IS_EMPTY(stack) // TRUE
PUSH(x)
[3,-1,-1,-1,-1]
PUSH(45)
[3,45,-1,-1,-1]
```

Trzeba wtedy pilnować jeszcze, czy starczy nam miejsca na więcej elementów.

Stos oparty na liście

PUSH - dodanie elementu na początek listy.

POP - usunięcie elementu z początku listy.

Można też dodawać/usuwać do końca listy (ale musimy się w implementacji zdecydować na JEDNĄ strategię, nie mieszać). Ten pierwszy sposób jest efektywniejszy.

Rozmiar - teoretycznie nieograniczony (w praktyce - tyle, ile mamy dostępnej pamięci dynamicznej).

Operowanie na stosie

Będziemy ćwiczyć obsługę stosu niezależnie od implementacji. Zakładamy, że mamy stos w jakiejś zmiennej np. **s** oraz zdefiniowane wszystkie potrzebne operacje **POP**, **PUSH**, **IS_EMPTY**. Podczas wykonywania algorytmów posługujących się stosem - będziemy śledzić stan stosu po zastosowaniu poszczególnych operacji.

PUSH(s,1)	4			3
PUSH(s,5)	5	POP()	PUSH(s,0)	0
-> PUSH(s,4)	-> 1	-> POP()-> 1	-> PUSH(s,3)	-> 1
s	s	s	s	s

Czubek stosu jest zawsze u góry rysunku. Na początku, **s** jest pusty - co widać w kolumnie najbardziej po lewej. Potem, po kolei, 3 operacje **PUSH()**, pisząc od góry do dołu - w takiej właśnie kolejności działają. Zatem najpierw na stos kładziemy 1, potem 5 i potem 4. Potem **POP()** zastosowany 2 razy usuwa 2 wartości z góry. I potem znowu mamy 2 instrukcje push, jedna po drugiej.

Drugi przykład, z wykorzystaniem pomocniczych zmiennych **a, b**:

PUSH(s,3)	4	a:=POP()	
-> PUSH(s,4)	-> 3	-> b:=POP()->	-> PUSH(s,a-b) -> 1
s	s	s	s

Na początku, stos pusty. Potem kładziemy 3, a potem 4. ściągamy to co jest na czubku i zapisujemy do a (czyli 4), potem znowu i zapisujemy do b (czyli mamy tam 3). Potem kładziemy **a-b**, co jest równe po prostu **1**.

Zastosowania

Najbardziej nam znane to np. śledzenie historii przeglądania witryn internetowych w przeglądarce. Każda szanująca się przeglądarka ma przyciski `UNDO` oraz `REDO`, odpowiednio cofające na poprzednie strony, lub "cofające cofnięcia". Potrzebne nam 2 odrębne stosy, `A` oraz `B`. Podczas przeglądania stron:

- z każdą nową zmianą strony, kładziemy wszystkie wczytane dane z tej strony na stos `A`.
- Jeśli naciśniemy `UNDO`, to: ściągamy stronę ze stosu `A` i kładziemy na stos `B`.
- Jeśli naciśniemy `REDO`, to: ściągamy stronę ze stosu `B` i kładziemy z powrotem na stos `A`.

(natomiast jak wykonamy sekwencję `UNDO`, `REDO` i potem odwiedzimy nową stronę - różnie się może życie potoczyć. Z mojego doświadczenia wynika, że wtedy stos `B` jest czyszczony.)

Odwrócona Notacja Polska (ONP) (*Reverse Polish Notation*)

Jedno z naturalnych zastosowań stosu. Zapis ten pozwala na jednoznacznie określenie kolejności wykonywania działań w wyrażeniu arytmetycznym, bez konieczności stosowania nawiasów. Ułatwia to zatem przekazanie informacji do komputera.

Zasada jest prosta: najpierw piszemy 2 wartości, a potem operator, jaki chcemy zastosować:

`2 2 +` to jest `4`

`3 2 /` to jest `1.5`

`4 0 *` to jest `0` itd.

Bardziej złożone przypadki:

`2 2 + 3 4 - *` to by było `(2+2)*(3-4)`

`1 1 2 + %` to będzie `1 % (1+2)`

Obliczenie bardziej złożonych przypadków jest jasne, jak zastosujemy algorytm ze stosem:

Obliczenie wartości wyrażenia w ONP:

Wejście: ciąg liczb i operatorów arytmetycznych opisujących działanie arytmetyczne,

Wyjście: obliczona wartość.

1. Zainicjalizuj stos `s` jako pusty.
2. Bierz po kolei elementy wejściowego ciągu:
 1. Jeśli element to liczba, połóż ją na stos.

2. Jeśli element to operator arytmetyczny:

1. ściągnij dwie liczby ze stosu i zapisz te wartości: `b:=POP(s)`; `a:=POP(s)`
2. Wstaw wynik działania na stos: `PUSH(a <operator> b)`, np. `PUSH(a+b)` jeśli operator to `+`, `PUSH(a-b)` jeśli operator to minus etc. UWAGA: liczba najbardziej na czubku (pierwsza ściągnięta) musi być po prawej stronie operatora.
3. Jak skończą się elementy ciągu wejściowego: zwróć ostatnią wartość na stosie `RETURN POP(s)`.

Algorytm MUSI skończyć na pustym stosie. Jeśli nie - coś sknocono w wejściowym ciągu liczb i operatorów.

Przykłady zastosowania (skrótowy zapis, nie piszemy `PUSH(s,2)` tylko `PUSH(2)`, bo mamy jeden stos `s` i wiadomo o co chodzi):

`2 2 + 3 4 - * :`

```
                b:=POP()                4 b:=POP()                b:=POP()
    PUSH(2)      2  a:=POP()      PUSH(3)  3 a:=POP()      -1  a:=POP()
->PUSH(2)-> 2  ->PUSH(a+b)-> 4 ->PUSH(4)-> 4->PUSH(a-b)-> 4 ->PUSH(a*b)-> -4
s                s                s                s                s
RETURN -4
```

Liczyliśmy po kolei $2+2$, potem $3-4$ i na koniec $4 * (-1)$ Pomijamy w zapisie ostatnią, oczywistą instrukcję `POP()`.

`1 1 2 + % : ( a % b` to reszta z dzielenia a przez b)

```
    PUSH(1)      2  b:=POP()                b:=POP()
    PUSH(1)      1  a:=POP()      3  a:=POP()
->PUSH(2)-> 1  ->PUSH(a+b)-> 1 ->PUSH(a*b) -> 1
s                s                s                s

RETURN 1
```

Najpierw $1+2$, potem $1 \% 3$, czyli ostatecznie 1.

Zadanie

Rozpisać na kartce wykonanie algorytmu obliczenia wyrażenia według ONP dla przydzielonego przykładu (sprawdzić, który dokładnie został przydzielony w mailu). Wypisać stany stosu, krok po

kroku działania algorytmu według podanych przykładów, aż do momentu wyliczenia końcowej wartości wyrażenia.

1) 2 4 4 1 9 * + % *

2) 1 9 - 9 4 - 6 + +

3) 9 2 % 1 1 4 - % *

4) 8 3 5 2 * 6 + - %

5) 5 7 % 1 % 3 + 1 -

6) 5 1 3 4 8 + * * %

7) 8 4 9 4 7 * - - *

8) 6 3 1 2 - * % 4 %

9) 7 2 % 3 * 2 - 3 *

10) 8 6 1 3 + - * 7 -